

LayerOneX

WASM to eBPF Compiler

VIRTUAL MACHINE

Security Audit

Platform

L1X

hashlock.com.au

February 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Behaviours	9
Code Quality	9
Audit Resources	9
Dependencies	10
Severity Definitions	10
Audit Findings	21
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The LayerOneX team partnered with Hashlock to conduct a security audit of their WASM to LLVM IR translator. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed infrastructure is secure.

Project Context

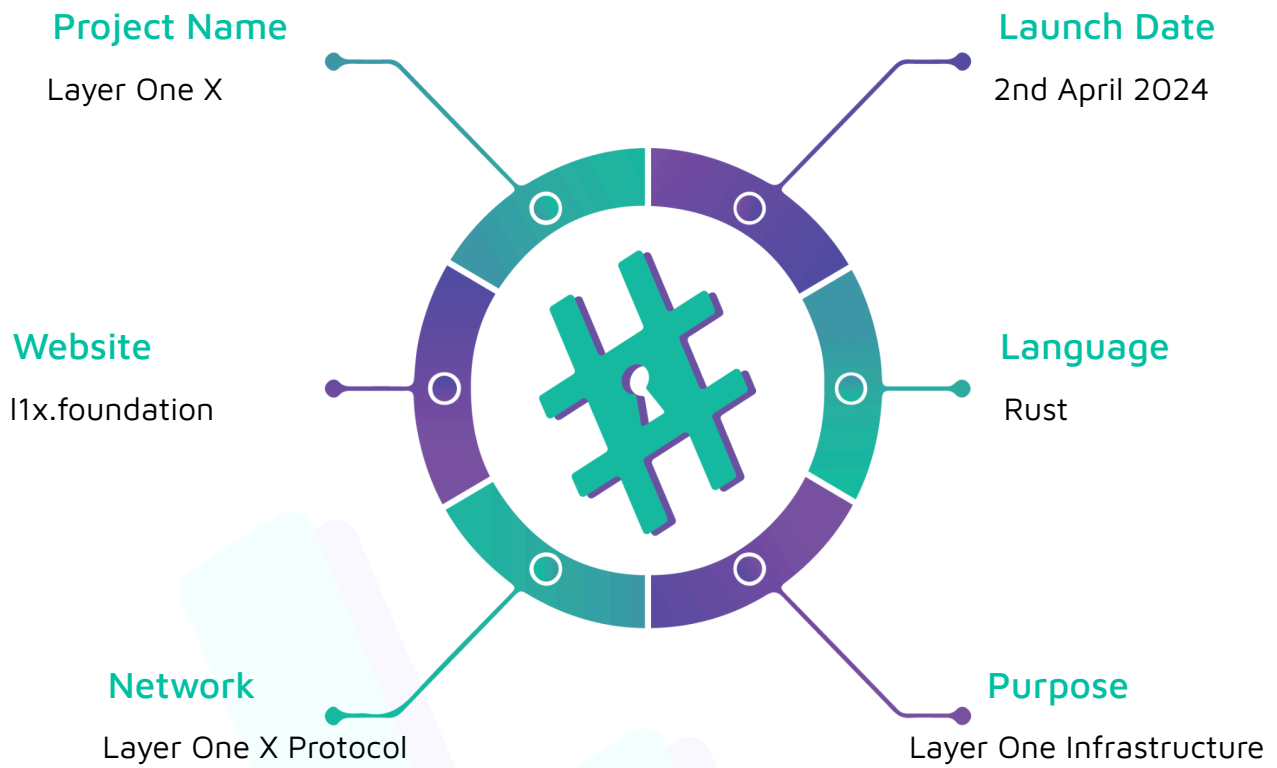
Layer One X was developed to embody the ideals of a truly decentralised, permissionless, and secure blockchain. It enables micro-validation and tokenization. The platform demonstrates how fast and scalable interoperability can deliver more equitable outcomes for everyone involved.

Project Name: Layer One X

Website: <https://www.l1x.foundation/>

Logo:



Visualised Context:

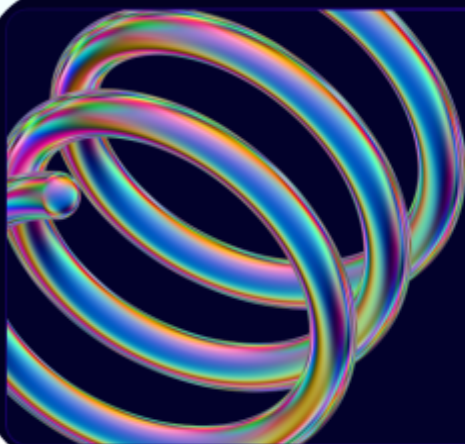
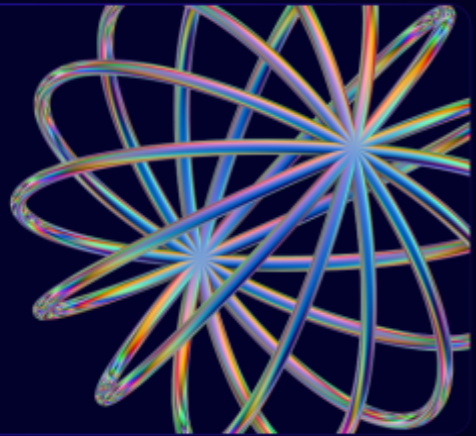
Project Visuals:



Interoperability

Greater interoperability with fewer integrations

Rapidly build dynamic smart contracts and dApps on Layer One X VM or EVM. Easily move data, digital assets, content and tokens across chains, wallets or digital worlds fast, securely and at low cost.



Scalability

Superior infrastructure for dApps that scale

Get scalability with almost instant finality. The Layer One X VM on Rust opens up the potential of executing more than 100,000 transactions per second with lower transaction fees.

#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the infrastructure and code within the LayerOneX project, the scope of works included a comprehensive review of the code files listed below. We tested the translator to check for its security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Scope 1

Description	L1X WASM to LLVM IR
Platform	L1X / Rust
Audit Date	February, 2024
File 1	l1x-wasm-llvmir/src/main.rs
File 1 MD5 Hash	2ea8fdf6816f860246514c6df4224c8d
File 2	l1x-wasm-llvmir/src/environment_impl.rs
File 2 MD5 Hash	2f135aa5d65b7a06585ce0c59dc881e5
GitHub Commit Hash	be6fab05d2158611e2b6df145ddb85ac

Scope 2

Description	
Platform	
Audit Date	
File 1	
File 1 MD5 Hash	
File 2	
File 2 MD5 Hash	
GitHub Commit Hash	

Security Rating

After Hashlock's Audit, we found the program to be **"Secure"**. The program follows simple logic, with correct and detailed ordering. The program relies on the inkwell and cranelift libraries.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The project's functionality is presented in the [Intended Behaviours](#) section.

Hashlock found:

Scope 1

0 High severity vulnerabilities

2 Medium severity vulnerabilities

9 Low severity vulnerabilities

Scope 2

x High severity vulnerabilities

x Medium severity vulnerabilities

x Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Behaviours

Claimed Behaviour	Actual Behaviour
main.rs, environment_impl.rs - Allows users to: - Translate WASM to LLVM IR	Program achieves this functionality.

Code Quality

This Audit scope involves the WASM to LLVM IR translator project, as outlined in the Audit Scope section. All code, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the WASM to LLVM IR translator code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this project are based on well known industry standard open source projects.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings - Scope 1

Medium

[M-01] `environment_impl` - Possible error scenarios are being silently ignored

Description

Some error indicating states are silently ignored when it could be advantageous to log an error.

Vulnerability Details

In `environment_impl.rs`:

```
match self {  
    ControlBlock::Function { results, .. } => {  
        results.append(&mut other);  
    }  
    _ => (),  
}  
  
...  
  
ControlBlock::IfElse {  
    ...  
} => {  
    let new_stack = self  
        .current_stack()  
        .copy_stack(params_count)  
        .expect("Can't copy value stack");  
    self.vec.push(ControlBlock::IfElse {  
        then,  
        if_else,
```

```

        end,
        else_is_found,
        value_stack: new_stack.clone(),
        params_count,
        results,
        stack_polymorphic: false,
    ))
}
_ => (),
}

...

let block_ref = self.top_mut().unwrap();
match block_ref {
    ControlBlock::IfElse { else_is_found, .. } => {
        *else_is_found = true;
    }
    _ => (),
}

```

Impact

If unexpected states do occur, it is best to handle that as an error.

Recommendation

Check for unexpected states instead of ignoring them.

Status

Acknowledged

[M-02] environment_impl - Unimplemented functionality is not marked as such

Description

Some functionality is not implemented yet not marked as such.



Vulnerability Details

In `environment_impl.rs`, these functions do nothing, but return success.

```
fn declare_memory_export(
    &mut self,
    _memory_index: MemoryIndex,
    _name: &'data str,
) -> WasmResult<()> {
    // self.info.memories[memory_index]
    //     .export_names
    //     .push(String::from(name));
    Ok(())
}
...
fn declare_global_export(
    &mut self,
    _global_index: GlobalIndex,
    _name: &'data str,
) -> WasmResult<()> {
    // self.info.globals[global_index]
    //     .export_names
    //     .push(String::from(name));
    Ok(())
}
```

Impact

These functions do nothing but still indicate success.

Recommendation

Make sure to mark unimplemented functionality as such. Implement if required.

Status

Acknowledged. Required by `cranelift` to return success.

Low

[L-01] `environment_impl` - Lack of documentation and code comments

Description

While there are some insightful code comments, there is a general lack of comments and documentations in this code base. This makes it harder to comprehend the reasoning behind certain decisions and decreases the maintainability of the code base.

Vulnerability Details

There is a lack of comments throughout the code base. This is a hindrance to the maintainability of the code base.

Recommendation

Write some documentation, especially for the reasoning behind decisions that may not be entirely obvious.

Status

Acknowledged

[L-02] `environment_impl` - Lack of error handling

Description

While it may be okay for development tools to abort and crash, it would be good to spend time improving error handling and providing helpful messages when aborting.

Vulnerability Details

There are 60 uses of `expect!()`, 27 uses of `unwrap!()`, 6 uses of `panic!()`.

Impact

Being too generous with `expect`, `unwrap`, `panic`, `todo!`, `unimplemented`, etc. speeds up development but makes this tool unstable.

Recommendation

Spend time to implement error handling and provide helpful information in case of errors.

Status

Acknowledged

[L-03] environment_impl - Code needs to be refactored**Description**

The code is essentially in one big file. This makes it hard to follow and reason about and thereby decreases the maintainability of the project.

Vulnerability Details

All the code is in environment_impl.rs. This file is about 3000 lines of which one function is ~1550 lines.

Impact

This code is very hard to read. This will make it harder to maintain the code base.

Recommendation

Split it into several logical files and break up excessively large functions.

Status

Acknowledged

[L-04] environment_impl - Drop function does not check length of vec**Description**

The drop function does not consider the length of the vec.

Vulnerability Details

In `environment_impl.rs`, the `drop` function does not check if there is any element that can be dropped.

```
fn drop(&mut self, count: usize) {  
    for _ in 0..count {  
        self.stack.pop();  
    }  
}
```

Impact

Nothing will be done as a `None` will be returned, but it should be considered if the attempt to do so qualifies as an error.

Recommendation

Make sure this functionality is implemented as desired.

Status

Acknowledged

[L-05] `environment_impl` - Commented out code

Description

There are several instances of commented out code, often without reasoning for the decision.

Vulnerability Details

In `environment_impl.rs`, there are some sections of commented out code. Some are marked `unimplemented` but others remain unclear.

```
// assert_eq!(  
//     self.info.functions.len(),  
//     self.info.imported_funcs.len(),  
//     "Imported functions must be declared first"
```

```
// );
// self.info.functions.push(Exportable::new(index));
// self.info
//     .imported_funcs
//     .push((String::from(module), String::from(field)));
...
// {
//     let bb = llvm_context.append_basic_block(llvm_function, "Body");
//     llvm_builder.build_unconditional_branch(bb);

//     llvm_builder.position_at_end(bb);
// }
```

Impact

Commented out code indicates unfinished functionality.

Recommendation

Verify these functions are implemented as designed.

Status

Acknowledged

[L-06] environment_impl - Code TODOs and unimplemented functionality

Description

There are many instances of unfinished code in this project.

Vulnerability Details

In environment_impl.rs, there are 11 instances of todo!()'s, 21 uses of unimplemented!() and several additional code notes:

```
// TODO: Uncomment when the eBPF machine will be able to relocate .rodata
// global_var.set_constant(!global.mutability);
...
```

```
assert!(fn_results.len() < 2, "TODO: Function can't return an array");
...
assert!(results.len() < 2, "TODO: Function can't return an array");
```

Impact

The code is unfinished in many places.

Recommendation

Implement the missing functionality.

Status

Acknowledged

[L-07] environment_impl - Global index ignored in initialization

Description

The global index provided for initialization is ignored.

Vulnerability Details

In `environment_impl.rs`, the global index is not used. Variables with global references are supposed to use that global for variable initialization.

```
if base.is_some() {
    // What `Global index` is? Need to initialize the given global variable with the
    // given data?
    todo!("What `Global index` is?");
}
```

Impact

The global variable is not used to initialize from. Functionality is not implemented.

Recommendation

Add functionality if required.

Status

Acknowledged

[L-08] environment_impl - Defining functions is incomplete

Description

This function appears to be unfinished as it pushes empty FnBody structs.

Vulnerability Details

In environment_impl.rs, only empty FnBody (function bodies) are pushed. Most of the code is commented out.

```
fn define_function_body(
    &mut self,
    validator: FuncValidator<ValidatorResources>,
    body: FunctionBody<'data>,
) -> WasmResult<()> {
    // self.func_bytecode_sizes
    //     .push(body.get_binary_reader().bytes_remaining());
    // let func = {
    //     let mut func_environ =
    //                                     DummyFuncEnvironment::new(&self.info,
self.expected_reachability.clone());
    //     let func_index =
    //                                     FuncIndex::new(self.get_num_func_imports() +
self.info.function_bodies.len());

    //     let sig = func_environ.vmctx_sig(self.get_func_type(func_index));
    //     let mut func =
    //                                     ir::Function::with_name_signature(UserFuncName::user(0,
func_index.as_u32()), sig);

    //     if self.debug_info {
    //         func.collect_debug_info();
    //     }
}
```

```

        //      self.trans
        //      .translate_body(&mut    validator,    body,    &mut    func,    &mut
func_environ)?;
        //      func
        // };
        // self.info.function_bodies.push(func);

        self.add_fn_param_buffer();
        self.add_math_op_syscalls();
        self.add_memory_op_syscalls();
        self.translate_function(validator, body)?;
        self.function_bodies.push(FnBody {});

        Ok(())
    }

```

Impact

This function is not fully implemented.

Recommendation

Make sure this function is correctly implemented.

Status

Acknowledged

[L-09] environment_impl - Use of unsafe code with build_gep

Description

This code uses unsafe functions with raw pointer manipulation.

Vulnerability Details

In environment_impl.rs, there are two instances of unsafe {} code calling build_gep().

Impact

Unsafe code may cause runtime exceptions.

Recommendation

Make sure to remove unsafe code once a safe version of the required functionality is available.

Status

Acknowledged

Audit Findings - Scope 2

Centralisation

The LayerOneX project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Layer One X project seems to have a sound and well tested code base, after the resolution of our findings. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#Hashlock.

#Hashlock.

Hashlock Pty Ltd